Cours Complet de JavaScript pour Débutants

Table des matières

1. Introduction et historique....8
    1.1 Qu'est-ce que JavaScript?....8
    1.2 Historique de JavaScript....8
    1.3 Pourquoi apprendre JavaScript?....8
2. Environnement de développement....8
    2.1 Outils nécessaires....9
    2.2 Configuration de base....9
        Créer votre premier script JavaScript....9
    2.3 Console JavaScript....9
    2.4 Exercice 1: Configuration de l'environnement....10
3. Bases de JavaScript....11
    3.1 Syntaxe de base....11
    3.2 Affichage et interaction....11
    3.3 Exercice 2: Interaction basique....12
4. Variables et types de données....13
    4.1 Déclaration de variables....13
    4.2 Types de données primitifs....13
    4.3 Conversion de types....14
    4.4 Vérification de types....15
    4.5 Exercice 3: Manipulation de variables et types....15
5. Opérateurs....16
5.1 Opérateurs arithmétiques....17
    Addition....17
    Soustraction....17
    Multiplication....17
    Division....17
    Modulo....18
    Exponentiation....18
5.2 Incrémentation et décrémentation....19
    Incrémentation....19
    Décrémentation....19
5.3 Pré-incrémentation et post-incrémentation....19
        Post-incrémentation....19
        Pré-incrémentation....20
5.4 Opérateurs d'affectation....20
    Affectation simple....20
    Affectations combinées....20

5.5 Opérateurs de comparaison **20**
Différence entre == et === 21
5.6 Opérateurs logiques **22**
Exemple 22
Opérateur OU 22
Opérateur NON 22
5.7 Opérateur ternaire **22**
5.8 Exercice : Calculateur simple **23**
Solution 23
6. Structures conditionnelles **23**
6.1 Instruction if **24**
6.2 if / else **24**
6.3 if / else if **24**
6.4 switch **25**
6.5 Bonnes pratiques **25**
6.6 Exercice : classification de notes **25**
7. Boucles **26**
7.1 Boucle for **26**
7.2 Boucle while **26**
7.3 Boucle do...while **27**
7.4 break **27**
7.5 continue **27**
7.6 Boucles imbriquées **27**
7.7 Exercice : table de multiplication **28**
Solution 28
8. Fonctions **28**
8.1 Déclaration d'une fonction **28**
8.2 Fonctions avec paramètres **29**
Plusieurs paramètres 29
8.3 Valeur de retour (return) **30**
8.4 Portée des variables (scope) **30**
locales 30
globales 30
8.5 Fonctions anonymes **31**
8.6 Fonctions fléchées (Arrow functions) **31**
8.7 Fonctions comme paramètres (callback) **31**
8.8 Exercice **32**
Solution 32
9. Tableaux **32**
9.1 Création d'un tableau **32**
9.2 Accéder à un élément **32**
9.3 Modifier un élément **33**
9.4 Taille d'un tableau **33**
9.5 Ajouter un élément **33**

push....33
unshift....33
9.6 Supprimer un élément.... 33
pop.... 33
shift....33
9.7 Parcourir un tableau.... 33
9.8 Exercice.... 34
Solution.... 34
10. Méthodes de tableaux....34
10.1 forEach....34
10.2 map....35
10.3 filter.... 35
10.4 reduce.... 35
10.5 find....36
10.6 includes....36
10.7 Exercice.... 36
11. Objets.... 36
11.1 Création d'un objet....37
11.2 Accès aux propriétés....37
11.3 Modifier une propriété.... 37
11.4 Ajouter une propriété....37
11.5 Méthodes d'objet....37
11.6 Parcourir un objet.... 38
11.7 Exercice....38
12. Manipulation du DOM.... 38
12.1 Qu'est-ce que le DOM ?....38
12.2 L'objet document.... 39
12.3 Sélection d'éléments.... 39
getElementById....39
querySelector.... 39
querySelectorAll.... 39
12.4 Modifier le contenu.... 39
textContent.... 39
innerHTML....39
12.5 Modifier les attributs....40
12.6 Modifier les styles....40
12.7 Ajouter ou supprimer des classes.... 40
12.8 Créer des éléments....40
12.9 Supprimer un élément.... 40
12.10 Exercice.... 40
13. Gestion des événements....41
13.1 addEventListener....41
Exemple.... 41
13.2 Événements souris.... 42

13.3 Événements clavier....42
13.4 Objet event....42
13.5 Propagation des événements....43
15. Gestion des formulaires....43
14.1 Structure d'un formulaire HTML....44
Attributs importants de <form>....44
14.2 Les principaux champs de formulaire....44
Champ texte....44
Champ email....45
Champ mot de passe....45
Zone de texte (textarea)....45
Liste déroulante....45
Boutons radio....45
Cases à cocher....45
14.3 Accéder aux données d'un formulaire en JavaScript....46
14.4 Empêcher l'envoi du formulaire....46
14.5 Récupérer la valeur des champs....46
14.6 Exemple complet....47
14.7 Validation des formulaires....47
Vérifier si un champ est vide....48
Vérifier la longueur d'un mot de passe....48
Vérifier un email avec une expression régulière....48
14.8 Afficher des messages d'erreur dans la page....48
14.9 Utiliser FormData....48
14.10 Réinitialiser un formulaire....49
14.11 Désactiver un bouton après envoi....49
14.12 Exemple complet de formulaire validé....49
14.13 Exercice 1 : Formulaire simple....50
Correction....51
14.14 Exercice 2 : Validation d'un formulaire....51
Correction....51
14.15 Points clés à retenir....52
15. Programmation asynchrone en JavaScript....52
15.1 Programmation synchrone vs asynchrone....53
Programmation synchrone....53
Problème avec les opérations longues....53
Programmation asynchrone....54
15.2 Le mécanisme interne : Event Loop....54
Call Stack....54
Web APIs....54
Callback Queue....55
Event Loop....55
15.3 setTimeout....55
Exemple....55

Exemple avec fonction nommée..... 55
Annuler un timeout..... 56
15.4 setInterval..... 56
Exemple..... 56
Arrêter un interval..... 56
15.5 Callbacks..... 56
Exemple simple..... 56
Exemple avec setTimeout..... 57
15.6 Le problème du Callback Hell..... 57
15.7 Les Promises..... 58
Création d'une Promise..... 58
15.8 Utilisation des Promises..... 58
then()..... 59
catch()..... 59
finally()..... 59
15.9 Chaînage de Promises..... 59
15.10 Async / Await..... 60
Déclaration d'une fonction async..... 60
Utilisation de await..... 60
15.11 Exemple complet..... 60
15.12 Gestion des erreurs avec async/await..... 61
15.13 Exercice 1..... 62
Correction..... 62
15.14 Exercice 2..... 62
Correction..... 62
15.15 Exercice 3 (async/await)..... 63
Correction..... 63
15.16 Points essentiels à retenir..... 63
16. Gestion des erreurs en JavaScript..... 63
16.1 Types d'erreurs en JavaScript..... 64
1. SyntaxError..... 64
2. ReferenceError..... 64
3. TypeError..... 65
4. RangeError..... 65
5. URIError..... 65
16.2 Détection des erreurs..... 65
16.3 Le bloc try...catch..... 65
Exemple..... 66
16.4 Accéder aux informations de l'erreur..... 66
Exemple..... 66
16.5 Le bloc finally..... 67
Exemple..... 67
16.6 Lancer une erreur avec throw..... 68
Exemple simple..... 68

Exemple avec Error....68
16.7 Création d'erreurs personnalisées....68
16.8 Gestion des erreurs dans les fonctions....69
16.9 Gestion des erreurs avec Promises....69
16.10 Gestion des erreurs avec async/await....70
16.11 Bonnes pratiques....70
1. Ne pas ignorer les erreurs....70
2. Donner des messages clairs....70
3. Utiliser des erreurs personnalisées....70
16.12 Exemple complet....70
16.13 Exercice 1....71
Correction....71
16.14 Exercice 2....72
Correction....72
16.15 Points essentiels à retenir....73
17. APIs et requêtes HTTP....73
17.1 Qu'est-ce qu'une API ?....73
17.2 Format JSON....74
17.3 Conversion JSON....74
Convertir un objet en JSON....74
Convertir JSON en objet....74
17.4 Les méthodes HTTP....75
17.5 La fonction fetch()....75
17.6 Exemple simple....75
17.7 Utilisation avec async / await....76
17.8 Envoyer des données avec POST....76
17.9 Vérifier la réponse HTTP....77
17.10 Exemple complet....77
17.11 Exercice....78
Correction....78
18. Stockage côté client....79
18.1 LocalStorage....79
18.2 Enregistrer une donnée....79
18.3 Lire une donnée....79
18.4 Supprimer une donnée....79
18.5 Vider le stockage....79
18.6 Stocker des objets....80
Enregistrement....80
Lecture....80
18.7 SessionStorage....80
18.8 Cookies....80
18.9 IndexedDB....81
18.10 Exemple pratique....81
18.11 Exercice....82

Correction....82
19. Concepts avancés JavaScript.... 82
19.1 Les closures.... 82
19.2 Le prototype.... 83
19.3 this....83
19.4 Destructuring....84
19.5 Spread operator.... 84
19.6 Rest parameters.... 84
19.7 Modules JavaScript.... 85
Export.... 85
Import.... 85
19.8 Bonnes pratiques....85
19.9 Exercice.... 86
Correction....86
20. Bonnes pratiques en JavaScript.... 86
20.1 Nommer correctement les variables.... 87
20.2 Utiliser la convention camelCase....87
20.3 Préférer const et let à var....87
20.4 Éviter les variables globales....88
20.5 Créer des fonctions petites et simples.... 88
20.6 Commenter le code intelligemment.... 88
20.7 Indentation et lisibilité.... 89
20.8 Éviter la duplication de code (DRY)....89
20.9 Validation des données.... 90
20.10 Gestion des erreurs.... 90
20.11 Organisation du projet....90
20.12 Utiliser des modules....91
20.13 Utiliser un linter....91
20.14 Formatter le code automatiquement....91
20.15 Exercice.... 92
Correction....92
21. Ressources et prochaines étapes....92
21.1 Projets recommandés....92
Débutant....92
Intermédiaire.... 93
Avancé....93
21.2 Frameworks et bibliothèques JavaScript.... 93
React....93
Vue.js....93
Angular....93
21.3 Outils importants.... 93
Node.js.... 94
npm.... 94
Git....94

21.4 Apprendre l'écosystème moderne..... 94
21.5 Lire la documentation officielle..... 94
21.6 Communautés et apprentissage.....95
21.7 Construire un portfolio..... 95
21.8 Contribuer à des projets open source..... 95
21.9 Continuer à apprendre.....96

1. Introduction et historique

1.1 Qu'est-ce que JavaScript?

JavaScript est un langage de programmation de haut niveau, interprété, qui permet d'ajouter de l'interactivité aux pages web. C'est un langage de script côté client, ce qui signifie qu'il s'exécute dans le navigateur de l'utilisateur, et non sur le serveur. JavaScript est l'un des trois piliers du développement web, avec HTML (structure) et CSS (présentation).

1.2 Historique de JavaScript

- **1995** : Créé par Brendan Eich chez Netscape en seulement 10 jours. Initialement appelé "Mocha", puis "LiveScript".
- **Décembre 1995** : Renommé "JavaScript" (pour des raisons marketing liées à la popularité de Java, bien que les deux langages soient très différents).
- **1997** : Standardisé par l'ECMA International sous le nom "ECMAScript".
- **1999** : ECMAScript 3 est publié et devient largement supporté.
- **2009** : ECMAScript 5 (ES5) est publié avec de nombreuses améliorations.
- **2015** : ECMAScript 6 (ES6/ES2015) apporte une refonte majeure du langage.
- **2016-présent** : Mises à jour annuelles (ES2016, ES2017, etc.) avec de nouvelles fonctionnalités.

1.3 Pourquoi apprendre JavaScript?

- **Omniprésence** : Présent dans presque tous les sites web modernes.
- **Polyvalence** : Utilisé côté client (navigateur) et côté serveur (Node.js).
- **Large écosystème** : Nombreuses bibliothèques et frameworks (React, Angular, Vue, Express, etc.).
- **Forte demande** : Compétence très recherchée sur le marché du travail.
- **Évolution constante** : Le langage continue de s'améliorer avec de nouvelles fonctionnalités.

2. Environnement de développement

2.1 Outils nécessaires

Pour commencer à coder en JavaScript, vous avez besoin de :

- Un navigateur web moderne (Chrome, Firefox, Edge, Safari)
- Un éditeur de texte ou IDE (Visual Studio Code, Sublime Text, Atom, WebStorm)

2.2 Configuration de base

Créer votre premier script JavaScript

1. Créez un dossier pour votre projet
2. Créez un fichier HTML (index.html) :

```html
<!DOCTYPE html>
<html lang="fr">
<head>
   <meta charset="UTF-8">
   <meta name="viewport" content="width=device-width, initial-scale=1.0">
   <title>Mon premier script JavaScript</title>
</head>
<body>
   <h1>Hello JavaScript!</h1>

   <!-- JavaScript interne -->
   <script>
      console.log("Bonjour depuis JavaScript interne!");
   </script>

   <!-- JavaScript externe -->
   <script src="script.js"></script>
</body>
</html>
```

3. Créez un fichier JavaScript (script.js) :

```javascript
console.log("Bonjour depuis un fichier JavaScript externe!");
alert("Bienvenue sur ma page!");
```

2.3 Console JavaScript

La console est un outil essentiel pour le développement JavaScript :

- Ouvrez-la en appuyant sur F12 ou clic droit > Inspecter > Console
- Utilisez-la pour :

- Déboguer votre code
- Tester rapidement des expressions
- Voir les erreurs et les messages de log

2.4 Exercice 1: Configuration de l'environnement

Objectif : Créer une page HTML avec un script JavaScript qui affiche des messages dans la console et une alerte.

Instructions :

1. Créez un fichier HTML nommé exercice1.html
2. Ajoutez la structure HTML de base
3. Incluez un script JavaScript interne qui affiche "Exercice 1 réussi!" dans la console
4. Créez un fichier JavaScript externe nommé exercice1.js
5. Dans ce fichier, écrivez du code qui affiche une alerte avec le texte "C'est mon premier script JavaScript!"
6. Liez le fichier externe à votre page HTML

Solution :

exercice1.html :

```html
<!DOCTYPE html>
<html lang="fr">
<head>
   <meta charset="UTF-8">
   <meta name="viewport" content="width=device-width, initial-scale=1.0">
   <title>Exercice 1 - JavaScript</title>
</head>
<body>
   <h1>Exercice 1 - Configuration de l'environnement</h1>

   <script>
      console.log("Exercice 1 réussi!");
   </script>

   <script src="exercice1.js"></script>
</body>
</html>
```

exercice1.js :

```javascript
alert("C'est mon premier script JavaScript!");
```

3. Bases de JavaScript

3.1 Syntaxe de base

JavaScript est sensible à la casse (maVariable et MaVariable sont différentes).

- **Instructions** : Se terminent par un point-virgule (facultatif mais recommandé)

```
console.log("Hello");
let a = 5;
```

- **Commentaires** : Pour documenter votre code

```
// Commentaire sur une ligne

/*
Commentaire
sur plusieurs
lignes
*/
```

- **Mots-clés réservés** : let, const, var, function, if, else, return, etc.

3.2 Affichage et interaction

- **console.log()** : Affiche des messages dans la console

```
console.log("Message simple");
console.log("Nombre:", 42);
console.log("Plusieurs", "arguments", "séparés");
```

- **alert()** : Affiche une boîte de dialogue

```
alert("Attention!");
```

- **prompt()** : Demande une saisie à l'utilisateur

```
let nom = prompt("Quel est votre nom?");
```

```
console.log("Bonjour, " + nom);
```

- **confirm()** : Demande une confirmation

```
let reponse = confirm("Êtes-vous sûr?");
if (reponse) {
   console.log("L'utilisateur a confirmé");
} else {
   console.log("L'utilisateur a annulé");
}
```

3.3 Exercice 2: Interaction basique

Objectif : Créer un script qui interagit avec l'utilisateur.

Instructions :

1. Créez un fichier HTML avec un script JavaScript
2. Le script doit :
    - Demander le nom de l'utilisateur via prompt()
    - Demander son âge via prompt()
    - Afficher un message personnalisé dans une alerte
    - Afficher les informations dans la console

Solution :

```
<!DOCTYPE html>
<html lang="fr">
<head>
   <meta charset="UTF-8">
   <meta name="viewport" content="width=device-width, initial-scale=1.0">
   <title>Exercice 2 - Interaction</title>
</head>
<body>
   <h1>Exercice 2 - Interaction basique</h1>

   <script>
      // Demander les informations
      let nom = prompt("Quel est votre nom?");
      let age = prompt("Quel est votre âge?");

      // Convertir l'âge en nombre
      age = parseInt(age);
```

```
    // Afficher un message personnalisé
    alert("Bonjour " + nom + "! Vous avez " + age + " ans.");

    // Afficher dans la console
    console.log("Nom:", nom);
    console.log("Âge:", age);
    console.log("L'année prochaine, vous aurez", age + 1, "ans.");
  </script>
</body>
</html>
```

4. Variables et types de données

4.1 Déclaration de variables

JavaScript propose trois façons de déclarer des variables :

- **let** : Pour les variables dont la valeur peut changer (recommandé)

```
let age = 25;
age = 26; // Possible de réaffecter
```

- **const** : Pour les constantes (valeurs qui ne changeront pas)

```
const PI = 3.14159;
// PI = 3.14; // Erreur! On ne peut pas réassigner une constante
```

- **var** : Ancienne façon (avant ES6, éviter si possible)

```
var nom = "Pierre";
```

4.2 Types de données primitifs

JavaScript comporte plusieurs types de données primitifs :

- **Number** : Nombres entiers et décimaux

```
let entier = 42;
let decimal = 3.14;
let negatif = -10;
```

```
let scientifique = 2.5e6; // 2.5 millions
```

- **String** : Chaînes de caractères

```
let prenom = "Marie";
let nom = 'Dupont';
let phrase = `Bonjour ${prenom}`; // Template literal (ES6)
```

- **Boolean** : Vrai ou faux

```
let estVrai = true;
let estFaux = false;
```

- **Undefined** : Variable déclarée mais sans valeur

```
let variable;
console.log(variable); // undefined
```

- **Null** : Absence intentionnelle de valeur

```
let vide = null;
```

- **Symbol** (ES6) : Valeur unique et immuable

```
let sym = Symbol("description");
```

- **BigInt** (Récent) : Pour les très grands nombres

```
let grandNombre = 9007199254740991n;
```

4.3 Conversion de types

JavaScript fait des conversions automatiques, mais vous pouvez les forcer :

```
// Conversion en nombre
let chiffre = "42";
let nombre = Number(chiffre); // 42
let nombreAlt = parseInt(chiffre); // 42
let decimal = parseFloat("3.14"); // 3.14

// Conversion en chaîne
let num = 42;
let chaine = String(num); // "42"
let chaineAlt = num.toString(); // "42"
```

```
// Conversion en booléen
let bool = Boolean(1); // true
```

4.4 Vérification de types

```
// Opérateur typeof
console.log(typeof 42); // "number"
console.log(typeof "bonjour"); // "string"
console.log(typeof true); // "boolean"
console.log(typeof undefined); // "undefined"
console.log(typeof null); // "object" (c'est un bug historique)
console.log(typeof []); // "object"
console.log(typeof {}); // "object"
```

4.5 Exercice 3: Manipulation de variables et types

Objectif : pratiquer la manipulation des variables et la conversion de types.

Instructions :

1. Déclarez une variable nombreTexte contenant la chaîne "42.5"
2. Convertissez cette chaîne en nombre et stockez-la dans nombreDecimal
3. Créez une variable arrondi contenant ce nombre arrondi à l'entier inférieur
4. Créez une variable booléenne estPositif qui vérifie si nombreDecimal est positif
5. Créez une variable message utilisant les template literals pour afficher toutes ces informations
6. Affichez le type de chaque variable avec typeof

Solution :

```
// Déclaration de la chaîne de caractères
let nombreTexte = "42.5";
console.log("nombreTexte:", nombreTexte, "Type:", typeof nombreTexte);

// Conversion en nombre
let nombreDecimal = parseFloat(nombreTexte);
console.log("nombreDecimal:", nombreDecimal, "Type:", typeof nombreDecimal);

// Arrondi à l'entier inférieur
let arrondi = Math.floor(nombreDecimal);
console.log("arrondi:", arrondi, "Type:", typeof arrondi);

// Vérification si positif
```

```
let estPositif = nombreDecimal > 0;
console.log("estPositif:", estPositif, "Type:", typeof estPositif);

// Création du message avec template literals
let message = `Le nombre "${nombreTexte}" converti donne ${nombreDecimal}.
Arrondi à l'entier inférieur: ${arrondi}.
Ce nombre est ${estPositif ? "positif" : "négatif"}.`;

console.log(message);
```

5. Opérateurs

Les opérateurs permettent d'effectuer **des opérations sur des valeurs ou des variables**.

Exemple simple :

let resultat = 5 + 3;

Dans cette expression :

- 5 et 3 sont des **opérandes**
- + est l'**opérateur**

JavaScript possède plusieurs catégories d'opérateurs :

- opérateurs arithmétiques
- opérateurs d'affectation
- opérateurs de comparaison
- opérateurs logiques
- opérateur ternaire
- opérateurs de chaînes
- opérateurs spéciaux

5.1 Opérateurs arithmétiques

Ces opérateurs permettent d'effectuer des **calculs mathématiques**.

Opérateur	Signification
+	addition
-	soustraction
*	multiplication
/	division
%	modulo
**	exponentiation

Addition

```
let a = 5;
let b = 3;

console.log(a + b);
```

Résultat :

8

Soustraction

```
console.log(10 - 4);
```

Résultat :

6

Multiplication

```
console.log(6 * 3);
```

Résultat :

18

Division

```
console.log(10 / 2);
```

Résultat :

5

Attention :

console.log(10 / 3);

Résultat :

3.333333333

JavaScript **ne fait pas automatiquement d'arrondi**.

Modulo

Le modulo renvoie **le reste d'une division**.

console.log(10 % 3);

Résultat :

1

Exemple d'utilisation : savoir si un nombre est **pair**.

```
let nombre = 8;

if (nombre % 2 === 0) {
  console.log("Nombre pair");
}
```

Exponentiation

Introduite avec ECMAScript 2016.

console.log(2 ** 3);

Résultat :

8

Equivalent :

Math.pow(2,3);

5.2 Incrémentation et décrémentation

Ces opérateurs servent à **augmenter ou diminuer une variable de 1**.

Incrémentation

```
let x = 5;

x++;

console.log(x);
```

Résultat :

6

Equivalent :

```
x = x + 1;
```

Décrémentation

```
let x = 5;

x--;

console.log(x);
```

Résultat :

4

5.3 Pré-incrémentation et post-incrémentation

Il existe deux formes :

Post-incrémentation

```
let x = 5;

console.log(x++);
```

Résultat :

5

Mais après :

x = 6

Pré-incrémentation

```
let x = 5;
```

console.log(++x);

Résultat :

6

5.4 Opérateurs d’affectation

Ils permettent d’affecter ou modifier une valeur.

Affectation simple

let x = 10;

Affectations combinées

let x = 10;

x += 5;

Equivalent :

x = x + 5;

Liste des opérateurs :

Opérateur	Equivalent
+=	x = x + y
-=	x = x - y
*=	x = x * y
/=	x = x / y
%=	x = x % y
**=	x = x ** y

5.5 Opérateurs de comparaison

Ils permettent de comparer des valeurs.

Opérateur	Signification

==	égalité
===	égalité stricte
!=	différent
!==	différent strict
>	supérieur
<	inférieur
>=	supérieur ou égal
<=	inférieur ou égal

Différence entre == et ===

```
let a = 5;
let b = "5";

console.log(a == b);
```

Résultat :

true

Car JavaScript **convertit automatiquement les types**.

Avec :

```
console.log(a === b);
```

Résultat :

false

Bonne pratique :

Toujours utiliser **===**.

5.6 Opérateurs logiques

Utilisés pour combiner plusieurs conditions.

Opérateur	Significations
&&	ET
||	OU
!	NON

Exemple

```
let age = 20;
let permis = true;

if (age >= 18 && permis) {
  console.log("Peut conduire");
}
```

Opérateur OU

```
let jour = "samedi";

if (jour === "samedi" || jour === "dimanche") {
  console.log("Weekend");
}
```

Opérateur NON

```
let connecte = false;

if (!connecte) {
  console.log("Veuillez vous connecter");
}
```

5.7 Opérateur ternaire

Syntaxe :

```
condition ? valeurSiVrai : valeurSiFaux
```

Exemple :

```
let age = 20;

let statut = age >= 18 ? "Majeur" : "Mineur";
```

Equivalent :

```
let statut;

if(age >= 18){
statut = "Majeur";
}else{
statut = "Mineur";
}
```

5.8 Exercice : Calculateur simple

Créer un programme qui :

1. demande deux nombres
2. affiche :
    - la somme
    - la multiplication
    - le plus grand nombre

Solution

```
let a = parseFloat(prompt("Premier nombre"));
let b = parseFloat(prompt("Deuxième nombre"));

let somme = a + b;
let produit = a * b;

let max = a > b ? a : b;

console.log("Somme :", somme);
console.log("Produit :", produit);
console.log("Plus grand :", max);
```

6. Structures conditionnelles

Les structures conditionnelles permettent d'exécuter **du code uniquement si une condition est vraie**.

6.1 Instruction if

Syntaxe :

```
if (condition) {

}
```

Exemple :

```
let age = 20;

if (age >= 18) {
   console.log("Vous êtes majeur");
}
```

6.2 if / else

Permet de gérer deux cas.

```
let age = 15;

if (age >= 18) {
   console.log("Majeur");
} else {
   console.log("Mineur");
}
```

6.3 if / else if

Pour gérer plusieurs cas.

```
let note = 14;

if (note >= 16) {
   console.log("Très bien");
}
else if (note >= 12) {
   console.log("Bien");
}
else if (note >= 10) {
   console.log("Passable");
}
else {
   console.log("Échec");
}
```

6.4 switch

Utilisé quand une variable peut avoir **plusieurs valeurs possibles**.

```
let jour = 2;

switch (jour) {

case 1:
console.log("Lundi");
break;

case 2:
console.log("Mardi");
break;

case 3:
console.log("Mercredi");
break;

default:
console.log("Jour inconnu");

}
```

Le mot-clé **break** est important pour éviter que les autres cas s'exécutent.

6.5 Bonnes pratiques

Préférer :

```
if(condition)
```

plutôt que :

```
if(condition == true)
```

6.6 Exercice : classification de notes

Créer un programme qui demande une note et affiche :

```
>=16  Excellent
>=12  Bien
>=10  Passable
<10   Échec
```

7. Boucles

Les boucles permettent de **répéter du code plusieurs fois**.

7.1 Boucle for

Structure :

```
for (initialisation; condition; incrémentation) {

}
```

Exemple :

```
for (let i = 0; i < 5; i++) {

console.log(i);

}
```

Résultat :

```
0
1
2
3
4
```

7.2 Boucle while

```
let i = 0;

while (i < 5) {

console.log(i);
i++;

}
```

7.3 Boucle do...while

La boucle s'exécute **au moins une fois**.

```
let i = 0;
```

```
do {

console.log(i);
i++;

} while (i < 5);
```

7.4 break

Permet de sortir d'une boucle.

```
for (let i = 0; i < 10; i++) {

if(i === 5){
break;
}

console.log(i);

}
```

7.5 continue

Permet de passer à l'itération suivante.

```
for (let i = 0; i < 5; i++) {

if(i === 2){
continue;
}

console.log(i);

}
```

7.6 Boucles imbriquées

Une boucle peut être placée dans une autre.

```
for (let i = 1; i <= 3; i++) {

for (let j = 1; j <= 3; j++) {

console.log(i, j);
```

```
}

}
```

7.7 Exercice : table de multiplication

Créer un programme qui affiche la table de multiplication d'un nombre.

Exemple pour 5 :

5 x 1 = 5
5 x 2 = 10
...
5 x 10 = 50

Solution

```
let nombre = parseInt(prompt("Donnez un nombre"));

for (let i = 1; i <= 10; i++) {

console.log(nombre + " x " + i + " = " + (nombre * i));
```

8. Fonctions

Une fonction est un **bloc de code réutilisable** qui effectue une tâche précise.

Sans fonctions, les programmes deviennent rapidement **répétitifs et difficiles à maintenir**.

8.1 Déclaration d'une fonction

Syntaxe :

```
function nomDeLaFonction() {

}
```

Exemple :

```
function direBonjour() {

console.log("Bonjour");
```

```
}
```

Pour exécuter la fonction :

```
direBonjour();
```

Résultat :

Bonjour

8.2 Fonctions avec paramètres

Les paramètres permettent de **passer des valeurs à la fonction**.

```
function direBonjour(nom) {

console.log("Bonjour " + nom);

}

direBonjour("Paul");
```

Résultat :

Bonjour Paul

Plusieurs paramètres

```
function addition(a, b) {

console.log(a + b);

}

addition(5,3);
```

Résultat :

8

8.3 Valeur de retour (return)

Une fonction peut **retourner une valeur**.

```
function addition(a,b){
```

```
return a + b;

}

let resultat = addition(4,6);

console.log(resultat);
```

Résultat :

10

⚠️ Dès que return est exécuté, la fonction **s'arrête immédiatement**.

8.4 Portée des variables (scope)

Les variables peuvent être :

locales

Accessibles uniquement dans la fonction.

```
function test(){

let x = 10;

console.log(x);

}
```

globales

Accessibles partout.

```
let x = 10;

function test(){

console.log(x);

}
```

8.5 Fonctions anonymes

Une fonction peut **ne pas avoir de nom**.

```
let saluer = function(){

console.log("Bonjour");

};

saluer();
```

8.6 Fonctions fléchées (Arrow functions)

Introduites dans ECMAScript 6.

Syntaxe :

```
const addition = (a,b) => {

return a + b;

};
```

Version courte :

```
const addition = (a,b) => a + b;
```

8.7 Fonctions comme paramètres (callback)

Les fonctions peuvent être passées **comme argument**.

```
function executer(operation){

operation();

}

executer(function(){

console.log("Fonction exécutée");

});
```

8.8 Exercice

Créer une fonction :

calculerMoyenne(note1,note2,note3)

qui retourne la moyenne.

Solution

```
function calculerMoyenne(a,b,c){

return (a + b + c) / 3;

}

let moyenne = calculerMoyenne(12,14,16);

console.log(moyenne);
```

9. Tableaux

Un tableau permet de **stocker plusieurs valeurs dans une seule variable**.

9.1 Création d'un tableau

```
let fruits = ["pomme","orange","mangue"];
```

Chaque valeur possède un **index**.

0 pomme
1 orange
2 mangue

9.2 Accéder à un élément

```
console.log(fruits[0]);
```

Résultat :

pomme

9.3 Modifier un élément

```
fruits[1] = "banane";
```

9.4 Taille d'un tableau

console.log(fruits.length);

9.5 Ajouter un élément

push

Ajoute à la fin.

fruits.push("ananas");

unshift

Ajoute au début.

fruits.unshift("fraise");

9.6 Supprimer un élément

pop

Supprime le dernier.

fruits.pop();

shift

Supprime le premier.

fruits.shift();

9.7 Parcourir un tableau

Avec une boucle :

```
for(let i = 0; i < fruits.length; i++){

console.log(fruits[i]);

}
```

9.8 Exercice

Créer un tableau contenant :

5 nombres

Afficher :

- la somme
- la moyenne

Solution

```
let nombres = [10,15,20,25,30];

let somme = 0;

for(let i=0;i<nombres.length;i++){

somme += nombres[i];

}

let moyenne = somme / nombres.length;

console.log("Somme :", somme);
console.log("Moyenne :", moyenne);
```

10. Méthodes de tableaux

Les tableaux possèdent **de nombreuses méthodes puissantes**.

10.1 forEach

Exécute une fonction pour chaque élément.

```
let nombres = [1,2,3];

nombres.forEach(function(nombre){

console.log(nombre);

});
```

10.2 map

Crée un **nouveau tableau transformé**.

```
let nombres = [1,2,3];

let doubles = nombres.map(function(n){

return n * 2;

});

console.log(doubles);
```

Résultat :

[2,4,6]

10.3 filter

Permet de **filtrer des valeurs**.

```
let nombres = [1,2,3,4,5];

let pairs = nombres.filter(function(n){

return n % 2 === 0;

});
```

Résultat :

[2,4]

10.4 reduce

Permet de **réduire un tableau à une seule valeur**.

Exemple : somme.

```
let nombres = [1,2,3,4];

let somme = nombres.reduce(function(total,n){

return total + n;

},0);
```

Résultat :

10

10.5 find

Trouve le premier élément correspondant.

```
let nombres = [10,20,30];

let resultat = nombres.find(n => n > 15);
```

Résultat :

20

10.6 includes

Vérifie si un élément existe.

```
let fruits = ["pomme","orange"];

console.log(fruits.includes("orange"));
```

Résultat :

true

10.7 Exercice

Créer un tableau :

[5,10,15,20,25]

Utiliser :

- filter pour garder les nombres > 10
- map pour les multiplier par 2

11. Objets

Les objets permettent de représenter **des entités complexes**.

11.1 Création d’un objet

```
let personne = {

nom: "Jean",
age: 30,
ville: "Paris"

};
```

11.2 Accès aux propriétés

```
console.log(personne.nom);
```

ou

```
console.log(personne["age"]);
```

11.3 Modifier une propriété

```
personne.age = 31;
```

11.4 Ajouter une propriété

```
personne.profession = "Développeur";
```

11.5 Méthodes d'objet

Un objet peut contenir des fonctions.

```
let personne = {

nom : "Paul",

saluer : function(){

console.log("Bonjour " + this.nom);

}

};

personne.saluer();
```

11.6 Parcourir un objet

```
for(let cle in personne){
```

```
console.log(cle, personne[cle]);

}
```

11.7 Exercice

Créer un objet :

Etudiant

avec :

nom
age
notes

Créer une fonction qui **calcule la moyenne**.

12. Manipulation du DOM

12.1 Qu’est-ce que le DOM ?

DOM signifie **Document Object Model**.

Le DOM est une **représentation en mémoire de la page HTML sous forme d’objets JavaScript**.

Exemple de HTML :

```
<h1>Titre</h1>
<p>Bonjour</p>
```

Dans le DOM cela devient une **structure d’objets** :

```
document
├── html
    ├── head
    └── body
        ├── h1
        └── p
```

Chaque élément HTML devient un **objet manipulable en JavaScript**.

12.2 L’objet document

L’objet principal pour manipuler la page est :

document

Exemples :

```
console.log(document.title)
console.log(document.body)
```

12.3 Sélection d’éléments

Pour manipuler un élément, il faut **le sélectionner**.

getElementById

```
<h1 id="titre">Bonjour</h1>
let titre = document.getElementById("titre")

console.log(titre)
```

querySelector

Permet d’utiliser **les sélecteurs CSS**.

```
let titre = document.querySelector("h1")
```

ou

```
document.querySelector("#titre")
```

querySelectorAll

Sélectionne **plusieurs éléments**.

```
let paragraphes = document.querySelectorAll("p")
```

12.4 Modifier le contenu

textContent

```
let titre = document.querySelector("h1")

titre.textContent = "Nouveau titre"
```

innerHTML

Permet d’insérer du HTML.

```
document.querySelector("div").innerHTML = "<b>Bonjour</b>"
```

⚠️ À utiliser avec prudence pour éviter des problèmes de sécurité.

12.5 Modifier les attributs

Exemple :

```
<img id="photo" src="image1.jpg">
let image = document.getElementById("photo")

image.src = "image2.jpg"
```

12.6 Modifier les styles

```
let titre = document.querySelector("h1")

titre.style.color = "red"
titre.style.fontSize = "40px"
```

12.7 Ajouter ou supprimer des classes

```
let element = document.querySelector("div")

element.classList.add("active")
element.classList.remove("active")
element.classList.toggle("active")
```

12.8 Créer des éléments

```
let p = document.createElement("p")

p.textContent = "Nouveau paragraphe"

document.body.appendChild(p)
```

12.9 Supprimer un élément

```
element.remove()
```

12.10 Exercice

Créer un bouton qui change le texte d’un titre.

HTML :

```
<h1 id="titre">Bonjour</h1>
<button id="btn">Changer</button>
```

JavaScript :

```
let bouton = document.getElementById("btn")

bouton.onclick = function(){

document.getElementById("titre").textContent = "Bonjour JavaScript"

}
```

13. Gestion des événements

Les événements permettent de **réagir aux actions de l'utilisateur**.

Exemples d'événements :

- clic
- mouvement de souris
- pression d'une touche
- soumission d'un formulaire

13.1 addEventListener

Méthode moderne pour écouter un événement.

```
element.addEventListener("typeEvenement", fonction)
```

Exemple

```
let bouton = document.querySelector("button")

bouton.addEventListener("click", function(){

console.log("Bouton cliqué")

})
```

13.2 Événements souris

événement	description
click	clic
dblclick	double clic
mouseover	souris sur élément
mouseout	souris quitte

Exemple :

```
let box = document.querySelector(".box")

box.addEventListener("mouseover", function(){

console.log("Souris dessus")

})
```

13.3 Événements clavier

événement	description
keydown	touche pressée
keyup	touche relâchée

Exemple :

```
document.addEventListener("keydown", function(event){

console.log(event.key)

})
```

13.4 Objet event

Chaque événement fournit des informations.

```
document.addEventListener("click", function(event){
```

```
console.log(event.target)

})
```

13.5 Propagation des événements

Deux phases :

1. **capturing**
2. **bubbling**

Dans la plupart des cas, JavaScript utilise **bubbling**.

15. Gestion des formulaires

Les formulaires HTML sont l'un des moyens principaux d'interaction entre un utilisateur et une application web. Ils permettent de collecter des données comme :

- nom
- email
- mot de passe
- choix dans une liste
- fichiers
- commentaires

JavaScript permet de :

- récupérer les données saisies
- vérifier leur validité
- empêcher l'envoi du formulaire si les données sont incorrectes
- envoyer les données vers un serveur (via API ou AJAX)

14.1 Structure d'un formulaire HTML

Un formulaire est défini avec la balise <form>.

Exemple simple :

```
<form id="monFormulaire">
  <label>Nom :</label>
  <input type="text" id="nom">

  <label>Email :</label>
  <input type="email" id="email">

  <button type="submit">Envoyer</button>
</form>
```

Attributs importants de <form>

Attribut	Description
action	URL vers laquelle envoyer les données
method	Méthode HTTP (GET ou POST)
id	Permet de sélectionner le formulaire en JavaScript

Exemple :

```
<form id="formulaire" action="/inscription" method="POST">
```

14.2 Les principaux champs de formulaire

Champ texte

```
<input type="text" id="nom">
```

Utilisé pour :

- nom
- prénom
- adresse

Champ email

```
<input type="email" id="email">
```

Le navigateur vérifie automatiquement si le format ressemble à un email.

Champ mot de passe

```
<input type="password" id="password">
```

Le texte est masqué.

Zone de texte (textarea)

Permet de saisir du texte long.

```
<textarea id="message"></textarea>
```

Liste déroulante

```
<select id="ville">
  <option value="paris">Paris</option>
  <option value="london">London</option>
  <option value="tokyo">Tokyo</option>
</select>
```

Boutons radio

Permettent de choisir **une seule option**.

```
<label>
<input type="radio" name="genre" value="homme"> Homme
</label>

<label>
<input type="radio" name="genre" value="femme"> Femme
</label>
```

Cases à cocher

Permettent **plusieurs choix**.

```
<label>
<input type="checkbox" value="javascript"> JavaScript
</label>

<label>
```

```
<input type="checkbox" value="python"> Python
</label>
```

14.3 Accéder aux données d’un formulaire en JavaScript

On commence par sélectionner le formulaire.

```
let formulaire = document.getElementById("monFormulaire");
```

Puis on écoute l'événement submit.

```
formulaire.addEventListener("submit", function(event){

  console.log("Formulaire envoyé");

});
```

14.4 Empêcher l'envoi du formulaire

Par défaut, un formulaire **rafraîchit la page** lorsqu’il est envoyé.

Pour empêcher cela :

```
event.preventDefault();
```

Exemple :

```
formulaire.addEventListener("submit", function(event){

  event.preventDefault();

  console.log("Envoi bloqué pour vérification");

});
```

14.5 Récupérer la valeur des champs

Exemple :

```
let nom = document.getElementById("nom").value;
let email = document.getElementById("email").value;

console.log(nom);
console.log(email);
```

La propriété .value permet de récupérer la valeur saisie.

14.6 Exemple complet

HTML :

```
<form id="formulaire">
  <input type="text" id="nom" placeholder="Nom">
  <input type="email" id="email" placeholder="Email">

  <button type="submit">Envoyer</button>
</form>
```

JavaScript :

```
let formulaire = document.getElementById("formulaire");

formulaire.addEventListener("submit", function(event){

  event.preventDefault();

  let nom = document.getElementById("nom").value;
  let email = document.getElementById("email").value;

  console.log("Nom :", nom);
  console.log("Email :", email);

});
```

14.7 Validation des formulaires

La validation permet de vérifier que les données saisies sont correctes.

Exemples :

- champ obligatoire
- email valide
- mot de passe assez long

Vérifier si un champ est vide

```
if(nom === ""){
```

```
  alert("Le nom est obligatoire");
}
```

Vérifier la longueur d'un mot de passe

```
if(password.length < 6){
  alert("Le mot de passe doit contenir au moins 6 caractères");
}
```

Vérifier un email avec une expression régulière

```
let regex = /^[^\s@]+@[^\s@]+\.[^\s@]+$/;

if(!regex.test(email)){
  alert("Email invalide");
}
```

14.8 Afficher des messages d'erreur dans la page

Au lieu d'utiliser alert, on peut afficher un message.

HTML :

```
<p id="erreur"></p>
```

JavaScript :

```
let erreur = document.getElementById("erreur");

erreur.textContent = "Veuillez remplir tous les champs";
```

14.9 Utiliser FormData

FormData permet de récupérer facilement toutes les données d'un formulaire.

```
let formData = new FormData(formulaire);

let nom = formData.get("nom");
let email = formData.get("email");
```

Avantage :

- simple
- fonctionne avec les fichiers

14.10 Réinitialiser un formulaire

formulaire.reset();

Exemple :

```
formulaire.addEventListener("submit", function(event){

  event.preventDefault();

  formulaire.reset();

});
```

14.11 Désactiver un bouton après envoi

Cela évite les **soumissions multiples**.

```
let bouton = document.querySelector("button");

bouton.disabled = true;
```

14.12 Exemple complet de formulaire validé

HTML :

```
<form id="formulaire">

<input type="text" id="nom" placeholder="Nom">

<input type="email" id="email" placeholder="Email">

<input type="password" id="password" placeholder="Mot de passe">

<button type="submit">S'inscrire</button>

<p id="message"></p>

</form>
```

JavaScript :

```
let form = document.getElementById("formulaire");
let message = document.getElementById("message");
```

```
form.addEventListener("submit", function(event){

event.preventDefault();

let nom = document.getElementById("nom").value;
let email = document.getElementById("email").value;
let password = document.getElementById("password").value;

if(nom === "" || email === "" || password === ""){
message.textContent = "Tous les champs sont obligatoires";
return;
}

if(password.length < 6){
message.textContent = "Mot de passe trop court";
return;
}

message.textContent = "Inscription réussie";

});
```

14.13 Exercice 1 : Formulaire simple

Objectif : récupérer les données d'un formulaire.

Instructions :

Créer un formulaire avec :

- nom
- âge
- ville

Au clic sur envoyer :

- empêcher le rechargement
- afficher les données dans la console

Correction

```
let form = document.getElementById("formulaire");

form.addEventListener("submit", function(event){

event.preventDefault();

let nom = document.getElementById("nom").value;
let age = document.getElementById("age").value;
let ville = document.getElementById("ville").value;

console.log(nom);
console.log(age);
console.log(ville);

});
```

14.14 Exercice 2 : Validation d'un formulaire

Créer un formulaire avec :

- email
- mot de passe

Règles :

- email obligatoire
- mot de passe minimum 6 caractères

Afficher les erreurs dans la page.

Correction

```
let form = document.getElementById("formulaire");
let message = document.getElementById("message");

form.addEventListener("submit", function(event){

event.preventDefault();

let email = document.getElementById("email").value;
let password = document.getElementById("password").value;
```

```
if(email === ""){
message.textContent = "Email obligatoire";
return;
}

if(password.length < 6){
message.textContent = "Mot de passe trop court";
return;
}

message.textContent = "Formulaire valide";

});
```

14.15 Points clés à retenir

- Les formulaires permettent de **collecter des données utilisateur**
- .value permet de récupérer la valeur d'un champ
- event.preventDefault() empêche le rechargement de la page
- La validation des données est essentielle
- FormData simplifie la récupération des données

15. Programmation asynchrone en JavaScript

Dans beaucoup d'applications, certaines opérations prennent du temps :

- charger des données depuis un serveur
- lire un fichier
- attendre une réponse réseau
- effectuer un calcul lourd
- attendre l'action d'un utilisateur

Si JavaScript exécutait ces opérations **de manière bloquante**, l'application serait figée.

Pour résoudre ce problème, JavaScript utilise un **modèle de programmation asynchrone**.

Cela signifie que certaines opérations peuvent être exécutées **sans bloquer le reste du programme**.

15.1 Programmation synchrone vs asynchrone

Programmation synchrone

Dans un modèle **synchrone**, les instructions sont exécutées **une par une dans l'ordre**.

Exemple :

```
console.log("Début")

console.log("Traitement")

console.log("Fin")
```

Résultat :

```
Début
Traitement
Fin
```

Chaque instruction attend que la précédente soit terminée.

Problème avec les opérations longues

Imaginons un téléchargement de données.

```
console.log("Début")

telechargerDonnees()

console.log("Fin")
```

Si telechargerDonnees() dure **5 secondes**, tout le programme est bloqué pendant ce temps.

Programmation asynchrone

Dans un modèle **asynchrone**, certaines opérations s'exécutent **en arrière-plan**.

Exemple :

```
console.log("Début")

setTimeout(function(){
console.log("Opération terminée")
},2000)

console.log("Fin")
```

Résultat :

```
Début
Fin
Opération terminée
```

JavaScript continue l'exécution sans attendre.

15.2 Le mécanisme interne : Event Loop

JavaScript fonctionne avec plusieurs composants :

- **Call Stack**
- **Web APIs**
- **Callback Queue**
- **Event Loop**

Call Stack

La pile d'exécution où les fonctions sont exécutées.

Web APIs

Fonctionnalités fournies par le navigateur :

- setTimeout

- fetch
- DOM events

Callback Queue

File d'attente des fonctions à exécuter.

Event Loop

Mécanisme qui surveille la Call Stack et la Callback Queue.

Quand la pile est vide, l'Event Loop prend une fonction dans la queue.

15.3 setTimeout

setTimeout() permet d'exécuter une fonction **après un certain délai**.

Syntaxe :

setTimeout(fonction, delai)

- délai en millisecondes
- 1000 ms = 1 seconde

Exemple

```
setTimeout(function(){
console.log("Bonjour après 3 secondes")
},3000)
```

Exemple avec fonction nommée

```
function direBonjour(){
console.log("Bonjour")
}

setTimeout(direBonjour,2000)
```

Annuler un timeout

```
let timer = setTimeout(function(){
console.log("Message")
},3000)
```

```
clearTimeout(timer)
```

15.4 setInterval

setInterval() exécute une fonction **à intervalles réguliers**.

Syntaxe :

```
setInterval(fonction, delai)
```

Exemple

```
setInterval(function(){
console.log("Toutes les secondes")
},1000)
```

Arrêter un interval

```
let interval = setInterval(function(){
console.log("Tick")
},1000)

clearInterval(interval)
```

15.5 Callbacks

Une **callback** est une fonction passée en argument à une autre fonction.

Elle est exécutée **plus tard**.

Exemple simple

```
function saluer(nom, callback){

console.log("Bonjour " + nom)

callback()

}

saluer("Paul", function(){
console.log("Bienvenue")
})
```

Résultat :

```
Bonjour Paul
```

Bienvenue

Exemple avec setTimeout

```
function operation(callback){

setTimeout(function(){

console.log("Opération terminée")

callback()

},2000)

}

operation(function(){
console.log("Suite du programme")
})
```

15.6 Le problème du Callback Hell

Lorsque plusieurs opérations asynchrones sont imbriquées, le code devient difficile à lire.

Exemple :

```
operation1(function(){

operation2(function(){

operation3(function(){

console.log("Terminé")

})

})

})
```

On appelle cela **Callback Hell**.

Pour résoudre ce problème, JavaScript a introduit **les Promises**.

15.7 Les Promises

Une **Promise** représente une opération qui sera terminée **dans le futur**.

Une Promise possède trois états :

État	Signification
pending	en attente
fulfilled	réussie
rejected	échouée

Création d'une Promise

```
let promesse = new Promise(function(resolve,reject){

let succes = true

if(succes){
resolve("Succès")
}else{
reject("Erreur")
}

})
```

15.8 Utilisation des Promises

Les Promises utilisent :

- .then()
- .catch()
- .finally()

then()

Exécuté si la promesse réussit.

```
promesse.then(function(resultat){

console.log(resultat)

})
```

catch()

Exécuté en cas d’erreur.

```
promesse.catch(function(erreur){

console.log(erreur)

})
```

finally()

Toujours exécuté.

```
promesse.finally(function(){

console.log("Fin de l'opération")

})
```

15.9 Chaînage de Promises

Les Promises peuvent être enchaînées.

```
fonction1()
.then(function(resultat){

return fonction2(resultat)

})
.then(function(resultat){

return fonction3(resultat)

})
.catch(function(erreur){
```

```
console.log(erreur)

})
```

15.10 Async / Await

async/await est une syntaxe moderne pour travailler avec les Promises.

Elle rend le code **plus lisible**.

Déclaration d'une fonction async

```
async function maFonction(){

return "Bonjour"

}
```

Une fonction async retourne toujours **une Promise**.

Utilisation de await

```
async function afficher(){

let resultat = await maFonction()

console.log(resultat)

}

afficher()
```

15.11 Exemple complet

Simulation d'une requête serveur.

```
function chargerDonnees(){

return new Promise(function(resolve){

setTimeout(function(){

resolve("Données chargées")

},2000)
```

```
})

}
```

Utilisation avec async/await :

```
async function afficher(){

console.log("Chargement...")

let donnees = await chargerDonnees()

console.log(donnees)

}

afficher()
```

Résultat :

```
Chargement...
(attente 2s)
Données chargées
```

15.12 Gestion des erreurs avec async/await

On utilise try...catch.

```
async function test(){

try{

let resultat = await operation()

console.log(resultat)

}catch(erreur){

console.log("Erreur :", erreur)

}

}
```

15.13 Exercice 1

Créer une fonction attendre qui affiche un message après 2 secondes.

Correction

```
function attendre(){

setTimeout(function(){

console.log("2 secondes écoulées")

},2000)

}

attendre()
```

15.14 Exercice 2

Créer une Promise qui simule un téléchargement de fichier.

Après 3 secondes, afficher :

Téléchargement terminé

Correction

```
function telecharger(){

return new Promise(function(resolve){

setTimeout(function(){

resolve("Téléchargement terminé")

},3000)

})

}

telecharger().then(function(message){

console.log(message)
```

```
})
```

15.15 Exercice 3 (async/await)

Réécrire l'exercice précédent avec async/await.

Correction

```
async function lancer(){

let message = await telecharger()

console.log(message)

}

lancer()
```

15.16 Points essentiels à retenir

- JavaScript utilise un **modèle asynchrone**
- setTimeout exécute une fonction après un délai
- setInterval répète une action
- les **callbacks** permettent d'exécuter du code plus tard
- les **Promises** améliorent la gestion de l'asynchrone
- **async/await** simplifie énormément l'écriture du code

16. Gestion des erreurs en JavaScript

Dans toute application, des erreurs peuvent se produire :

- une variable n'existe pas
- une fonction reçoit un mauvais argument
- une requête réseau échoue
- une donnée n'est pas valide

La **gestion des erreurs** permet de :

- détecter les problèmes
- empêcher le programme de planter
- afficher des messages utiles
- maintenir la stabilité de l'application

16.1 Types d'erreurs en JavaScript

JavaScript distingue plusieurs catégories d'erreurs.

1. SyntaxError

Erreur de syntaxe dans le code.

Exemple :

let x = ;

Résultat :

SyntaxError: Unexpected token ;

Le code ne peut pas être exécuté.

2. ReferenceError

Se produit lorsqu'on utilise une variable qui n'existe pas.

console.log(age)

Résultat :

ReferenceError: age is not defined

3. TypeError

Se produit lorsqu'une opération est faite sur un type incompatible.

let nombre = 10

nombre()

Résultat :

TypeError: nombre is not a function

4. RangeError

Une valeur est hors des limites acceptables.

let tableau = new Array(-5)

5. URIError

Erreur liée à l'encodage d'URL.

decodeURI("%")

16.2 Détection des erreurs

Sans gestion d'erreur, le programme s'arrête immédiatement.

Exemple :

console.log("Début")

erreur()

console.log("Fin")

Résultat :

Début
ReferenceError

La ligne "Fin" n'est jamais exécutée.

16.3 Le bloc try...catch

JavaScript permet d'intercepter les erreurs avec **try...catch**.

Syntaxe :

try{

// code à surveiller

}catch(erreur){

```
// gestion de l'erreur

}
```

Exemple

```
try{

let x = y

}catch(error){

console.log("Une erreur est survenue")

}
```

16.4 Accéder aux informations de l’erreur

L’objet error contient plusieurs propriétés.

propriété	description
name	type d’erreur
message	message
stack	trace d’appel

Exemple

```
try{

let x = y

}catch(error){

console.log(error.name)
console.log(error.message)

}
```

Résultat :

ReferenceError

y is not defined

16.5 Le bloc finally

finally est exécuté **dans tous les cas**, erreur ou non.

Syntaxe :

```
try{

// code

}catch(error){

// erreur

}finally{

// toujours exécuté

}
```

Exemple

```
try{

console.log("Essai")

}catch(error){

console.log("Erreur")

}finally{

console.log("Fin")

}
```

16.6 Lancer une erreur avec throw

On peut créer ses propres erreurs.

Syntaxe :

```
throw nouvelleErreur
```

Exemple simple

```
function verifierAge(age){

if(age < 18){

throw "Age insuffisant"

}

}

verifierAge(16)
```

Exemple avec Error

```
throw new Error("Age invalide")
```

16.7 Création d’erreurs personnalisées

On peut créer ses propres classes d’erreurs.

```
class ValidationError extends Error{

constructor(message){

super(message)

this.name = "ValidationError"

}

}
```

Utilisation :

```
throw new ValidationError("Nom obligatoire")
```

16.8 Gestion des erreurs dans les fonctions

Exemple :

```
function diviser(a,b){
```

```
if(b === 0){
throw new Error("Division par zéro")
}
return a / b
}
```

Utilisation :

```
try{
let resultat = diviser(10,0)
}catch(error){
console.log(error.message)
}
```

16.9 Gestion des erreurs avec Promises

Une Promise peut être **rejetée**.

```
let promesse = new Promise(function(resolve,reject){
reject("Erreur serveur")
})
```

Gestion :

```
promesse.catch(function(erreur){
console.log(erreur)
})
```

16.10 Gestion des erreurs avec async/await

On utilise try...catch.

```
async function charger(){
```

```
try{

let response = await fetch("url")

}catch(error){

console.log("Erreur :",error)

}

}
```

16.11 Bonnes pratiques

1. Ne pas ignorer les erreurs

Mauvais exemple :

```
try{

code()

}catch(e){}
```

2. Donner des messages clairs

```
throw new Error("Le mot de passe doit contenir 8 caractères")
```

3. Utiliser des erreurs personnalisées

Cela permet une meilleure gestion dans les applications complexes.

16.12 Exemple complet

```
function creerUtilisateur(nom){

if(!nom){

throw new Error("Nom obligatoire")

}

return {
```

```
nom: nom

}

}

try{

let user = creerUtilisateur("Paul")

console.log(user)

}catch(error){

console.log(error.message)

}
```

16.13 Exercice 1

Créer une fonction racineCarree.

Si le nombre est négatif, lancer une erreur.

Correction

```
function racineCarree(n){

if(n < 0){

throw new Error("Nombre négatif interdit")

}

return Math.sqrt(n)

}

try{

console.log(racineCarree(-4))

}catch(error){
```

```
console.log(error.message)

}
```

16.14 Exercice 2

Créer une fonction retirerArgent(solde, montant).

Conditions :

- montant > solde → erreur
- montant négatif → erreur

Correction

```
function retirerArgent(solde,montant){

if(montant < 0){

throw new Error("Montant invalide")

}

if(montant > solde){

throw new Error("Solde insuffisant")

}

return solde - montant

}

try{

let nouveauSolde = retirerArgent(100,200)

console.log(nouveauSolde)

}catch(error){

console.log(error.message)
```

```
}
```

16.15 Points essentiels à retenir

- les erreurs sont inévitables dans les programmes
- try...catch permet d'intercepter les erreurs
- finally est toujours exécuté
- throw permet de créer des erreurs
- les Promises utilisent catch
- async/await utilise try...catch

17. APIs et requêtes HTTP

Les applications modernes doivent souvent **échanger des données avec des serveurs**.

Exemples :

- récupérer des utilisateurs depuis une base de données
- envoyer un formulaire
- afficher des produits d'un site e-commerce
- récupérer la météo

Ces communications se font grâce au **protocole HTTP** et aux **APIs**.

17.1 Qu'est-ce qu'une API ?

Une **API (Application Programming Interface)** est une interface qui permet à deux applications de communiquer.

Exemples :

- application météo → API météo

- application bancaire → API paiement
- application mobile → serveur backend

Une API renvoie généralement les données au format **JSON**.

17.2 Format JSON

JSON signifie **JavaScript Object Notation**.

C'est un format texte utilisé pour échanger des données.

Exemple :

```
{
"name": "Paul",
"age": 30,
"ville": "Paris"
}
```

En JavaScript, cela correspond à un objet :

```
let utilisateur = {
name: "Paul",
age: 30,
ville: "Paris"
}
```

17.3 Conversion JSON

Convertir un objet en JSON

```
let user = {nom:"Paul", age:30}

let json = JSON.stringify(user)

console.log(json)
```

Résultat :

```
{"nom":"Paul","age":30}
```

Convertir JSON en objet

```
let data = '{"nom":"Paul","age":30}'
```

```
let obj = JSON.parse(data)

console.log(obj.nom)
```

17.4 Les méthodes HTTP

Lorsqu'un client communique avec un serveur, il utilise une **méthode HTTP**.

méthode	utilisation
GET	récupérer des données
POST	envoyer des données
PUT	modifier des données
DELETE	supprimer des données

17.5 La fonction fetch()

JavaScript fournit la fonction **fetch()** pour effectuer des requêtes HTTP.

Syntaxe :

```
fetch(url)
```

Elle retourne une **Promise**.

17.6 Exemple simple

```
fetch("https://jsonplaceholder.typicode.com/users")
.then(function(response){

return response.json()

})
.then(function(data){

console.log(data)

})
```

Étapes :

1. envoyer la requête
2. recevoir la réponse
3. convertir en JSON
4. utiliser les données

17.7 Utilisation avec async / await

La syntaxe moderne :

```
async function chargerUtilisateurs(){

let response = await fetch("https://jsonplaceholder.typicode.com/users")

let utilisateurs = await response.json()

console.log(utilisateurs)

}

chargerUtilisateurs()
```

17.8 Envoyer des données avec POST

Exemple :

```
fetch("https://api.example.com/users",{

method: "POST",

headers:{
"Content-Type":"application/json"
},

body: JSON.stringify({
nom:"Paul",
age:30
})

})
```

17.9 Vérifier la réponse HTTP

Un serveur peut renvoyer différents statuts.

code	signification
200	succès
201	créé
400	requête invalide
404	ressource introuvable
500	erreur serveur

Exemple :

```
async function charger(){

let response = await fetch("url")

if(!response.ok){

throw new Error("Erreur serveur")

}

}
```

17.10 Exemple complet

```
async function chargerPosts(){

try{

let response = await fetch("https://jsonplaceholder.typicode.com/posts")

let posts = await response.json()

posts.forEach(function(post){

console.log(post.title)

})
```

```
}catch(error){

console.log("Erreur :",error)

}

}

chargerPosts()
```

17.11 Exercice

Créer un programme qui :

1. récupère les utilisateurs depuis l'API :

https://jsonplaceholder.typicode.com/users

2. affiche leurs noms dans la console.

Correction

```
async function afficherUtilisateurs(){

let response = await fetch("https://jsonplaceholder.typicode.com/users")

let users = await response.json()

users.forEach(function(user){

console.log(user.name)

})

}

afficherUtilisateurs()
```

18. Stockage côté client

Les applications web ont souvent besoin de **conserver des données dans le navigateur**.

Exemples :

- préférences utilisateur
- panier d'achat
- thème sombre
- session utilisateur

Le navigateur fournit plusieurs systèmes de stockage.

18.1 LocalStorage

LocalStorage permet de stocker des données persistantes dans le navigateur.

Caractéristiques :

- stockage clé/valeur
- persistant (même après fermeture du navigateur)
- capacité ≈ 5MB

18.2 Enregistrer une donnée

```
localStorage.setItem("nom","Paul")
```

18.3 Lire une donnée

```
let nom = localStorage.getItem("nom")

console.log(nom)
```

18.4 Supprimer une donnée

```
localStorage.removeItem("nom")
```

18.5 Vider le stockage

```
localStorage.clear()
```

18.6 Stocker des objets

LocalStorage ne stocke que **des chaînes**.

On utilise JSON.

Enregistrement

```
let utilisateur = {

nom:"Alice",
age:28

}

localStorage.setItem("user",JSON.stringify(utilisateur))
```

Lecture

```
let data = localStorage.getItem("user")

let utilisateur = JSON.parse(data)

console.log(utilisateur.nom)
```

18.7 SessionStorage

Fonctionne comme LocalStorage mais :

- les données disparaissent lorsque l'onglet est fermé

Exemple :

```
sessionStorage.setItem("token","12345")
```

18.8 Cookies

Les **cookies** permettent aussi de stocker des données.

Ils sont souvent utilisés pour :

- authentification
- suivi utilisateur

- préférences

Créer un cookie :

```
document.cookie = "user=Paul"
```

18.9 IndexedDB

IndexedDB est une **base de données dans le navigateur**.

Caractéristiques :

- stockage structuré
- grandes quantités de données
- accès asynchrone

Utilisé dans :

- applications offline
- PWA
- stockage d’images

18.10 Exemple pratique

Sauvegarder un thème sombre.

```
localStorage.setItem("theme","dark")
```

Puis au chargement :

```
let theme = localStorage.getItem("theme")

if(theme === "dark"){

document.body.classList.add("dark")

}
```

18.11 Exercice

Créer un programme qui :

1. demande un nom utilisateur
2. le sauvegarde dans LocalStorage
3. l'affiche au rechargement de la page.

Correction

```
let nom = prompt("Entrez votre nom")

localStorage.setItem("nom",nom)

let sauvegarde = localStorage.getItem("nom")

console.log("Bienvenue " + sauvegarde)
```

19. Concepts avancés JavaScript

Ce chapitre couvre des notions importantes pour maîtriser JavaScript.

19.1 Les closures

Une **closure** est une fonction qui se souvient des variables de son environnement.

Exemple :

```
function compteur(){

let count = 0

return function(){

count++

console.log(count)

}

}

let incrementer = compteur()
```

```
incrementer()
incrementer()
incrementer()
```

Résultat :

```
1
2
3
```

19.2 Le prototype

JavaScript utilise un système **basé sur les prototypes**.

Chaque objet possède un prototype.

Exemple :

```
function Personne(nom){

this.nom = nom

}

Personne.prototype.saluer = function(){

console.log("Bonjour " + this.nom)

}

let p = new Personne("Paul")

p.saluer()
```

19.3 this

this représente l'objet courant.

Exemple :

```
let personne = {

nom:"Paul",
```

```
saluer:function(){

console.log("Bonjour " + this.nom)

}

}

personne.saluer()
```

19.4 Destructuring

Permet d’extraire des valeurs d’un objet.

```
let utilisateur = {

nom:"Alice",
age:25

}

let {nom,age} = utilisateur

console.log(nom)
```

19.5 Spread operator

Permet de copier ou fusionner des objets.

```
let a = {x:1}
let b = {y:2}

let c = {...a,...b}

console.log(c)
```

19.6 Rest parameters

Permet de recevoir un nombre variable d’arguments.

```
function somme(...nombres){

let total = 0
```

```
nombres.forEach(function(n){

total += n

})

return total

}

console.log(somme(1,2,3,4))
```

19.7 Modules JavaScript

Les modules permettent de **diviser le code en fichiers**.

Export

```
export function addition(a,b){

return a + b

}
```

Import

```
import {addition} from "./math.js"

console.log(addition(2,3))
```

19.8 Bonnes pratiques

- utiliser const par défaut
- éviter les variables globales
- découper le code en fonctions
- utiliser les modules
- gérer les erreurs

19.9 Exercice

Créer une fonction createCounter utilisant une closure.

Correction

```
function createCounter(){

let count = 0

return function(){

count++

return count

}

}

let counter = createCounter()

console.log(counter())
console.log(counter())
console.log(counter())
```

20. Bonnes pratiques en JavaScript

Savoir écrire du code qui fonctionne est une chose.
Savoir écrire **du code propre, maintenable et robuste** est une compétence essentielle.

Les bonnes pratiques permettent :

- de rendre le code lisible
- de réduire les bugs
- de faciliter la maintenance
- de permettre le travail en équipe

20.1 Nommer correctement les variables

Les noms de variables doivent être **clairs et explicites**.

❌ Mauvais :

```
let x = 25
let a = "Paul"
```

✅ Bon :

```
let ageUtilisateur = 25
let nomUtilisateur = "Paul"
```

20.2 Utiliser la convention camelCase

En JavaScript, on utilise généralement **camelCase**.

Exemples :

```
let nombreUtilisateurs
let prixTotal
let dateCreation
```

Pour les constantes :

```
const MAX_UTILISATEURS = 100
```

20.3 Préférer const et let à var

var est l'ancienne façon de déclarer les variables.

Aujourd'hui :

- **const** → valeur qui ne change pas
- **let** → valeur modifiable

Exemple :

```
const PI = 3.14
let compteur = 0
```

Éviter :

```
var x = 10
```

20.4 Éviter les variables globales

Les variables globales peuvent provoquer des conflits.

❌ Mauvais :

```
let total = 0

function ajouter(){
total++
}
```

✅ Bon :

```
function ajouter(total){

return total + 1

}
```

20.5 Créer des fonctions petites et simples

Une fonction doit idéalement **faire une seule chose**.

❌ Mauvais :

```
function traiterCommande(){

calculerPrix()

envoyerEmail()

mettreAJourStock()

}
```

✅ Bon :

```
function calculerPrix(){}

function envoyerEmail(){}

function mettreAJourStock(){}
```

20.6 Commenter le code intelligemment

Les commentaires doivent expliquer **le pourquoi**, pas le quoi.

❌ Mauvais :

```
// ajouter 1
compteur++
```

✅ Bon :

```
// incrémente le compteur après chaque connexion utilisateur
compteur++
```

20.7 Indentation et lisibilité

Un code bien indenté est plus lisible.

❌ Mauvais :

```
if(age>18){
console.log("majeur")}
```

✅ Bon :

```
if(age > 18){

console.log("majeur")

}
```

20.8 Éviter la duplication de code (DRY)

Principe **DRY : Don't Repeat Yourself**

❌ Mauvais :

```
let total1 = prix1 + taxe
let total2 = prix2 + taxe
let total3 = prix3 + taxe
```

✅ Bon :

```
function calculerTotal(prix){

return prix + taxe

}
```

20.9 Validation des données

Toujours vérifier les données entrées par l’utilisateur.

Exemple :

```
function calculerAge(annee){

if(typeof annee !== "number"){

throw new Error("année invalide")

}

return 2025 - annee

}
```

20.10 Gestion des erreurs

Toujours anticiper les erreurs.

```
try{

let data = JSON.parse(text)

}catch(error){

console.log("JSON invalide")

}
```

20.11 Organisation du projet

Un projet JavaScript doit être bien structuré.

Exemple :

```
projet/
│
├── index.html
├── css/
│   └── style.css
│
├── js/
```

```
│   ├── app.js
│   ├── api.js
│   └── utils.js
│
└── assets/
```

20.12 Utiliser des modules

Les modules permettent de **séparer les responsabilités**.

math.js

```
export function addition(a,b){
return a + b
}
```

app.js

```
import {addition} from "./math.js"

console.log(addition(2,3))
```

20.13 Utiliser un linter

Un **linter** analyse automatiquement le code.

Exemple :

- ESLint

Il permet de détecter :

- erreurs
- mauvaises pratiques
- incohérences

20.14 Formatter le code automatiquement

Outils populaires :

- Prettier

- ESLint formatter

Ils permettent d'avoir **un style de code uniforme**.

20.15 Exercice

Refactoriser ce code :

```
var x=10
var y=20
var z=x+y
console.log(z)
```

Correction

```
const nombre1 = 10
const nombre2 = 20

const resultat = nombre1 + nombre2

console.log(resultat)
```

21. Ressources et prochaines étapes

Après ce cours, l'étudiant possède **les bases solides de JavaScript**.

Pour devenir développeur expérimenté, il faut :

- pratiquer
- construire des projets
- apprendre les outils de l'écosystème

21.1 Projets recommandés

Pour progresser, construire des projets réels :

Débutant

- calculatrice
- gestionnaire de tâches (Todo list)

- convertisseur de devises

Intermédiaire

- application météo
- blog simple
- application de notes

Avancé

- application chat
- e-commerce simple
- gestionnaire de projet

21.2 Frameworks et bibliothèques JavaScript

Une fois les bases maîtrisées, apprendre un framework.

Les plus populaires :

React

Utilisé pour :

- interfaces dynamiques
- applications web modernes

Vue.js

Framework simple et puissant.

Angular

Framework complet utilisé pour les grandes applications.

21.3 Outils importants

Les développeurs JavaScript utilisent plusieurs outils.

Node.js

Permet d’exécuter JavaScript côté serveur.

npm

Gestionnaire de packages JavaScript.

Exemple :

npm install axios

Git

Système de gestion de version.

Commandes importantes :

```
git init
git add .
git commit -m "première version"
```

21.4 Apprendre l’écosystème moderne

Compétences importantes :

- ES6+
- modules JavaScript
- programmation asynchrone
- APIs REST
- Node.js
- frameworks frontend

21.5 Lire la documentation officielle

La documentation officielle est une compétence essentielle.

Ressources importantes :

- MDN Web Docs
- documentation ECMAScript
- documentation Node.js

21.6 Communautés et apprentissage

Participer aux communautés permet de progresser plus vite.

Plateformes utiles :

- GitHub
- Stack Overflow
- forums de développeurs

21.7 Construire un portfolio

Pour devenir développeur professionnel :

Créer un **portfolio avec plusieurs projets**.

Exemples :

- application météo
- gestionnaire de tâches
- application API
- site personnel

21.8 Contribuer à des projets open source

Contribuer à l’open source permet :

- d’apprendre
- de collaborer
- d’améliorer son CV

21.9 Continuer à apprendre

JavaScript évolue régulièrement.

Il faut suivre :

- nouvelles versions ECMAScript
- nouveaux frameworks
- nouvelles bonnes pratiques